# Ansible

# Introduction & installation

**Pourquoi Ansible ?**

- C'est une application open source gratuite
- Sans agent – Pas besoin d’installation et de gestion d’agent
- Basé sur Python / Yaml
- Gestion très flexible et de la configuration des systèmes
- Grand nombre de modules prêts à l'emploi pour la gestion du système
- Des modules personnalisés peuvent être ajoutés si nécessaire
- Restauration de la configuration en cas d'erreur
- Simple et lisible
- Auto-documentation

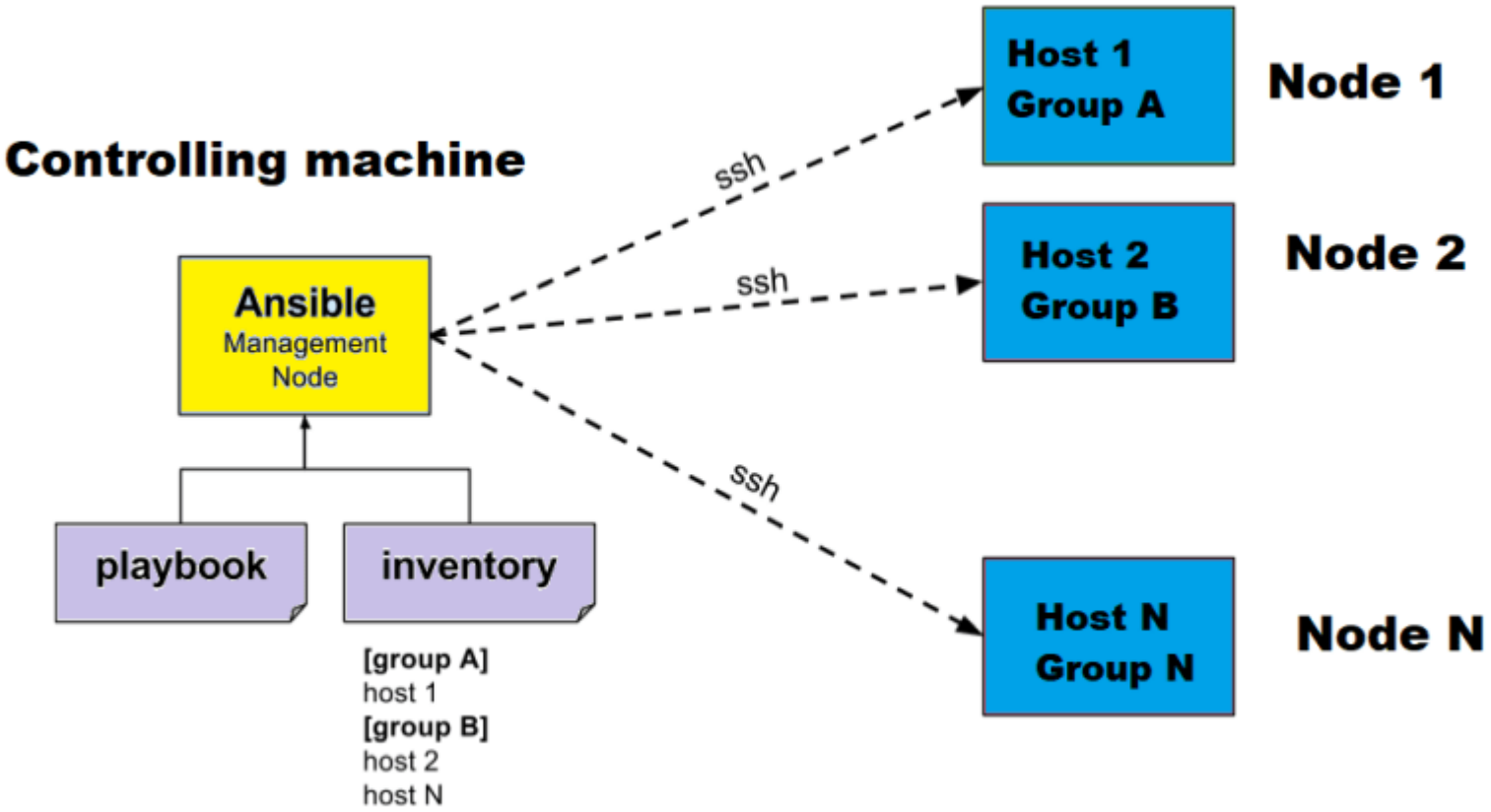
## D'où commencer Ansible

- Ansible est accompagné par une documentation accès riche sous https://docs.ansible.com/

- La section Get Started permet de se familiariser avec Ansible elle contient également une rubrique qui montre la procédure d'installation de Ansible https://docs.ansible.com/ansible/latest/installation_guide/intro_installation.html#installing-and-upgrading-ansible

- La documentation est egalement utilisée comme référenciel pour savoir comment utiliser les modules qui servent à automatiser certaines tâches

# Architecture de Ansible



## Les notions de Ansible

- **Le nœud de contrôle:** Machine à partir de laquelle vous exécutez les outils Ansible. Plusieurs nœuds de contrôle sont possibles dans ce contexte

- **Les nœuds gérés:** Appelé également hôtes, il s'agit des périphériques cibles (serveurs, appareils réseau ou tout ordinateur) que vous souhaitez gérer avec Ansible

- **L'inventaire:** Inventaire présente les informations spécifiques à chaque nœud, comme l'adresse IP. Il sert également à définir des variables utilisées ensuite selon les groupe

  L'inventaire peut être définit en format INI ou en format YAML

```
[ubuntu]
192.168.1.16
[ubuntu:vars]
valeur=8080
```

**Format INI**

```
all:
  hosts:
    ubuntu:
      ansible_host: 192.168.1.16
  vars:
    valeur: 8080
```

**Format YAML**

**Les notions de Ansible**

**Note:** Les inventaires sont définis par défaut sous **/etc/ansible/hosts** au niveau de la machine serveur.
Au cas du changement d'emplacement du fichier hosts, il faut bien indiquer le chemin avec l'option –i au niveau de la commande ansible Playbook

```
ansible –i chemin_du_fichier_hosts  -m nom_du_module
ansible-playbook –i chemin_du_fichier_hosts  nom_du_playbook
```

## Les notions de Ansible

- **Les commandes Ad hoc:** Ce sont des lignes de commandes lancées à partir du neoud de contrôle

- **Les playbooks:** Les Playbooks sont des fichiers YAML qui définissent une série de tâches à exécuter sur un ensemble de machines cibles

- **Les rôles:** Les rôles sont une façon d'organiser et de partager le code dans Ansible. Ils regroupent des Playbooks, des fichiers de variables, des tâches et des handlers pour simplifier la réutilisation et la modularité du code

- **Les collections:** Une collection Ansible est un ensemble regroupé de contenus Ansible, comprenant des rôles, des modules, des playbooks, des plugins et d'autres artefacts associés, généralement organisés autour d'un thème ou d'une fonctionnalité spécifique, elles sont crée par la commande **ansible-galaxy collection init nom_de_collection** une fois tout les éléments sont ajouté la collection sera construite avec **ansible-galaxy collection build** et puis elle sera publié **ansible-galaxy collection publish**

## Les notions de Ansible

- **Les tâches:** Les tâches représentent les actions spécifiques que vous souhaitez effectuer sur les machines cibles, Elles sont définies dans les Playbooks et peuvent inclure une variété d'opérations, telles que l'installation de logiciels, la modification de fichiers de configuration, le redémarrage de services, etc

- **Les modules:** Les modules sont des programmes Python exécutés sur les machines distantes par Ansible. Ils sont responsables de l'exécution des tâches définies dans les Playbooks, comme la gestion des packages, des services, des fichiers, etc.

```
- name: Installer le package nginx
  apt:
    name: nginx
    state: present
```

Exemple de tâche

Exemple de module

Les tâches sont exécutées dans l'ordre défini dans le Playbook, et Ansible assure que chaque tâche est exécutée uniquement si nécessaire pour amener le système à l'état spécifié dans le Playbook

## Les notions de Ansible

- **Les handlers:** Les handlers sont des tâches spéciales déclenchées uniquement si d'autres tâches dans le Playbook ont effectivement modifié l'état du système. Ils sont souvent utilisés pour redémarrer des services ou effectuer d'autres actions similaires

- **Les Facts:** Les facts sont des informations collectées par Ansible sur les machines cibles lors de l'exécution des Playbooks.

- **Les variables:** Les variables sont des valeurs dynamiques utilisées pour personnaliser le comportement des Playbooks. Elles peuvent être définies au niveau de

  1. L'inventaire
  2. Des Playbooks
  3. Des rôles

- **Les templates:** Les templates sont des fichiers de modèle utilisés pour générer des fichiers de configuration sur les machines cibles. Ils permettent d'intégrer des variables et des structures de contrôle dans les fichiers de configuration pour une personnalisation flexible. Les Template sont définies avec Jinja 2

## Les notions de Ansible

- **Le Vault:** Vault est un mécanisme intégré de gestion des secrets dans Ansible. Il permet de chiffrer et de déchiffrer des données sensibles telles que les mots de passe, les clés SSH et les jetons d'API, garantissant ainsi leur sécurité lors de leur stockage et de leur transmission.

- **Les plugins:** Sont des morceaux de code écrit en python, qui étendent les fonctionnalités de base d'Ansible en permettant aux utilisateurs d'ajouter des fonctionnalités personnalisées ou de modifier le comportement par défaut d'Ansible

# Pré installationAnsible

**Préparation de l'environnement Ansible**

- Pré installation côté serveur

  adduser ansadmin

  ssh-keygen -t rsa

  ssh-copy-id -i /home/ansadmin/.ssh/id_rsa.pub ansadmin@IP

- Pré installation côté client

  adduser ansadmin

  /etc/sudoers <= ansadmin ALL=(ALL) NOPASSWD:ALL

**Note:** Vérifier le paramètres **PasswordAuthentication** à **yes** au niveau de **/etc/ssh/sshd_config** au niveau du client, si non il faut le reconfigurer à **yes** et appliquer les changements via la commande **systemctl reload sshd**

- Installation Ansible

  apt install ansible –y

- Post installation côté serveur

  Editer le fichier inventaire /etc/ansible/hosts et ajouter les clients exemple

```
[clients]
        172.31.4.210
```

# Installation Ansible

**Installation de Ansible**

- Installation Ansible côté serveur

```
sudo apt update && sudo apt upgrade –y
sudo apt install -y software-properties-common
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt install -y ansible
ansible --version
```

- Fichier inventaire

```
sudo mkdir -p /etc/ansible
cd /etc/ansible
sudo vi /etc/ansible/hosts
```

# Dé installation Ansible

## Installation de Ansible

- Installation Ansible côté serveur

```
sudo apt remove --purge -y ansible
sudo apt autoremove –y
sudo add-apt-repository --remove ppa:ansible/ansible –y
sudo rm -rf /etc/ansible
```

# Les commandes Ansible

## Les commandes Ansible

**ansible:** Cette commande est utilisée pour exécuter une seule tâche sur un ou plusieurs hôtes gérés, elle est appelée également commande Ad hoc.

**ansible all -m ping**

**ansible-playbook:** Cette commande est utilisée pour exécuter des playbooks Ansible, qui sont des scripts qui contiennent une série de tâches à exécuter sur des hôtes gérés.

**ansible-playbook playbook.yml**

**ansible-galaxy:** Cette commande est utilisée pour créer, installer et gérer des rôles Ansible.

**ansible-galaxy init mon_rôle**

**ansible-doc:** Cette commande est utilisée pour afficher la documentation des modules Ansible comme le module copy par exemple.

**ansible-doc copy**

## Les commandes Ansible

**ansible-console:** Cette commande lance une console interactive Ansible.

**ansible-console**

**ansible-vault:** Cette commande est utilisée pour chiffrer, déchiffrer et éditer des fichiers contenant des données sensibles avec Ansible Vault.

**ansible-vault create secrets.yml**

**ansible-config:** Cette commande est utilisée pour gérer la configuration d'Ansible.

**ansible-config view**

**Remarque:** Si cette commande affiche une erreur, il va falloir créer un fichier ansible.cfg sous /etc/ansible et l'attribuer à la variable ANSIBLE_CONFIG
export ANSIBLE_CONFIG=/chemin/vers/votre/ansible.cfg

## La commande "ansible" ou la commande ad hoc

- Elle permet d’exécuter des tâches ad-hoc sur un ou plusieurs hôtes.
- Les commandes Ad hoc sont idéales pour les tâches rapides ou les opérations ponctuelles
- La commande ansible à des options

**-m ou --module-name :** Cette option est utilisée pour spécifier le module à utiliser. Par exemple, pour vérifier si tous les hôtes sont accessibles, vous pouvez utiliser le module ping

```
ansible all -m ping
```

**-a ou --args :** Cette option est utilisée pour passer des arguments au module. Par exemple, pour créer un répertoire sur tous les hôtes, vous pouvez utiliser le module file avec l’argument path et state :

```
ansible all -m file -a "path=/path/to/directory state=directory"
```

## La commande "ansible" ou la commande ad hoc

**-i ou --inventory :** Cette option est utilisée pour spécifier le fichier d'inventaire. Par exemple :

**ansible  -i /path/to/inventory -m ping**

**-u ou --user :** Cette option est utilisée pour définir l'utilisateur distant. Par exemple :

**ansible all -u remote_user -m ping**

**-b ou --become :** Cette option est utilisée pour obtenir les privilèges de superutilisateur. Par exemple, pour mettre à jour tous les paquets sur tous les hôtes, vous pouvez utiliser le module apt :

**ansible all -b -m apt -a "name=* state=latest"**

**-v ou --verbose :** Cette option est utilisée pour augmenter le niveau de verbosité. Vous pouvez utiliser jusqu'à 4 "v", par exemple :

**ansible all -vvvv -m ping**

## La commande "ansible" ou la commande ad hoc

**-C ou --check :** Cette option est utilisée pour effectuer un **dry run**, c’est-à-dire exécuter la commande sans apporter de modifications réelles. Par exemple :

**ansible all -C -m file -a "path=/path/to/directory state=directory"**

**Les commandes Ad hoc de Ansible**

- Voici quelques exemples de commandes ad-hoc

```
ansible all -m shell -a "df -h"
ansible webserver -m shell -a "ls -l /var/log"
ansible all -m command -a "reboot" --limit all
ansible all -m copy -a "src=/home/Ubuntu/fichier.txt dest=/home/ubuntu"
ansible all -m user -a "user=ubuntu2 password=test"
ansible all -m yum -a "name=httpd state=latest"
ansible all -m service -a "name=httpd state=started"
ansible all -m service -a "name=httpd state=stopped"
```

# Les playbooks ansible

## Les playbooks de ansible (Exemple installation du serveur nginx)

- Un playbook Ansible est un fichier de configuration, généralement écrit au format YAML, qui définit un ensemble de tâches à exécuter sur un ou plusieurs hôtes cibles.

- La commande **ansible-playbook** est utilisée pour exécuter des playbooks Ansible, qui sont des scripts contenant une série de tâches à exécuter sur des hôtes gérés. Voici quelques options fréquemment utilisées

**-i ou --inventory :** Cette option est utilisée pour spécifier le fichier d'inventaire. Par exemple :

**ansible-playbook -i /chemin/vers/inventaire playbook.yml**

**-u ou --user :** Cette option est utilisée pour définir l'utilisateur distant. Par exemple :

**ansible-playbook -u utilisateur playbook.yml**

**-k ou --ask-pass :** Cette option est utilisée pour demander un mot de passe SSH. Par exemple :

**ansible-playbook -k playbook.yml**

## Les playbooks de ansible (Exemple installation du serveur nginx)

**-b ou --become :** Cette option est utilisée pour obtenir les privilèges de superutilisateur. Par exemple :

**ansible-playbook -b playbook.yml**

**-v ou --verbose :** Cette option est utilisée pour augmenter le niveau de verbosité. Vous pouvez utiliser jusqu'à 4 "v", par exemple :

**ansible-playbook -vvvv playbook.yml**

**-C ou --check :** Cette option est utilisée pour effectuer un dry run, c'est-à-dire exécuter la commande sans apporter de modifications réelles. Par exemple :

**ansible-playbook -C playbook.yml**

## Les playbooks de ansible (Exemple installation du serveur nginx)

**--list-tasks :** Cette option est utilisée pour lister toutes les tâches qui seraient exécutées. Par exemple :

```
ansible-playbook --list-tasks playbook.yml
```

**--start-at-task :** Cette option est utilisée pour commencer le playbook à la tâche correspondant à ce nom. Par exemple :

```
ansible-playbook --start-at-task="Nom de la tâche" playbook.yml
```

## Les playbooks de ansible (Exemple installation du serveur nginx)

**Voir github:** https://github.com/bejaouibechir/Ansible/tree/main/Nginx%20Case

```
ansuser@PC2023:~$ ansible-playbook web.yml

PLAY [Setup web servers] ***********************

TASK [Gathering Facts] *************************
ok: [192.168.1.14]

TASK [Ensure Nginx is installed] ***************
ok: [192.168.1.14]

TASK [Ensure Nginx is running] *****************
ok: [192.168.1.14]

TASK [Copy index.html] *************************
changed: [192.168.1.14]

PLAY RECAP *************************************
192.168.1.14               : ok=4    changed=1
```

**Le résultat attendu**

## Les playbooks de ansible (déinstallation du serveur nginx)

**Voir github:** https://github.com/bejaouibechir/Ansible/tree/main/Nginx%20Case

```
ansadmin@machine1:~/playbooks$ ansible-playbook -b  nginx_uninstall.yml

PLAY [Uninstall Nginx] *********************************************************

TASK [Gathering Facts] *********************************************************
ok: [192.168.1.29]

TASK [Stop Nginx service if running] *******************************************
ok: [192.168.1.29]

TASK [Uninstall Nginx package] *************************************************
ok: [192.168.1.29]

PLAY RECAP *********************************************************************
192.168.1.29               : ok=3    changed=1    unreachable=0    failed=0    skipped=0
    rescued=0    ignored=0
```

**Le résultat attendu**

## Les Playbook de Ansible (Execution avec privilège élevée)

- Il existe deux manières pour lancer l'execution du playbook avec des privilèges élevé

  En ajoutant l'option –b au moment d'appel du Playbook
  **ansible-playbook -b web.yml**

  En ajoutant la ligne **become: true** à l'intérieur du Playbook

```yaml
---
- name: Setup web servers
  hosts: ubuntu
  become: true
  tasks:
    - name: Ensure Nginx is installed
      apt:
        name: nginx
        state: present

    - name: Ensure Nginx is running
      service:
        name: nginx
        state: started

    - name: Copy index.html
      copy:
        src: index.html
        dest: /var/www/html/
```

**Les Playbook de Ansible (Execution avec privilège élevée)**

**Note:** Il faut bien distinguer entre l'option **become** et **become_user**, la première commande élève le privilège pendant que la deuxième indique quel est l'utilisateur consterné par ce privilège

Ce code par exemple assure que l'utilisateur passe au privilège élevé pour exécuter le module git

```
---
- name: Setup web server
  hosts: all
  become: true
  tasks:
    - name: Ensure git is installed
      apt:
        name: git
        state: present
```

**Les Playbook de Ansible (Execution avec privilège élevée)**

Ce code assure que l'utilisateur passe au privilège élevé pour exécuter le module git et que l'utilisateur concerné est bien ansadmin

```
---
- name: Setup web server
  hosts: all
  become: true
  become_user: ansadmin
  tasks:
    - name: Ensure git is installed
      apt:
        name: git
        state: present
```

Note1 : Pour exécuter le Playbook à partir d'un autre utilisateur il faut configurer le certificat id_rsa/id_rsa.pub

Note2: Exécuter la commande ansible-playbook avec l'option –K pour entrer le mot de passe

Note3: Pour éviter l'option –K il faut ajouter l'utilisateur à la liste des sudoers avec
**ALL=(ALL:ALL) NOPASSWD:ALL**

# La configuration ansible

**La commande ansible-config "configuration de ansible**

- La commande **ansible-config** est utilisée pour gérer la configuration d'Ansible.

- Elle permet de voir tous les paramètres de configuration disponibles, leurs valeurs par défaut, comment les définir et d'où proviennent leurs valeurs actuelles

Voici quelques exemples d'utilisation de la commande ansible-config :

Afficher la configuration actuelle d'Ansible :

**ansible-config dump**

Afficher la valeur d'un paramètre de configuration spécifique :

**ansible-config dump --only-changed**

Vérifier la syntaxe du fichier de configuration d'Ansible :

**ansible-config check**

**La commande ansible-config "configuration de ansible**

- Les données de configuration sont stockées par défaut sous **/etc/ansible/ansible.cfg** sinon il est possible de changer cet endroit par défaut soit au même endroit que le **Playbook** ou le répertoire home de l'utilisateur

- Il est possible également de définir une variable globale **ANSIBLE_CONFIG** qui détient le chemin par défaut du fichier de configuration

- Voici quelques paramètres à prendre en considération  qui peuvent être stockés dans le fichier **ansible.cfg**

**inventory :** Ce paramètre définit le chemin vers votre fichier d'inventaire Ansible. Si vous avez un fichier d'inventaire que vous utilisez fréquemment, vous pouvez définir ce paramètre pour éviter de devoir spécifier l'option -i chaque fois que vous exécutez une commande Ansible

**[defaults]**
**inventory = /chemin/vers/votre/inventaire**

## La commande ansible-config "configuration de ansible

**forks :** Ce paramètre contrôle le nombre de processus parallèles qu'Ansible utilisera. Augmenter ce nombre peut accélérer l'exécution de vos playbooks si vous gérez un grand nombre d'hôtes

**[defaults]**
**forks = 20**

**remote_user :** Ce paramètre définit l'utilisateur par défaut pour les connexions SSH. Si vous vous connectez toujours en tant que même utilisateur, vous pouvez définir ce paramètre pour éviter de devoir spécifier l'option -u chaque fois que vous exécutez une commande Ansible

**[defaults]**
**remote_user = ansible**

## La commande ansible-config "configuration de ansible

**become** : Ce paramètre contrôle si Ansible doit obtenir les privilèges de superutilisateur par défaut. Si vous exécutez toujours des commandes en tant que superutilisateur, vous pouvez définir ce paramètre pour éviter de devoir spécifier l'option -b chaque fois que vous exécutez une commande Ansible

**[defaults]**
**become = True**

**roles_path :** Ce paramètre définit les chemins où Ansible cherchera les rôles. C'est utile si vous avez des rôles réutilisables que vous voulez garder dans un emplacement centralisé

**[defaults]**
**roles_path = /chemin/vers/vos/rôles**

**log_path :** Ce paramètre définit le chemin du fichier de journalisation d'Ansible. Si vous voulez garder une trace de ce que fait Ansible, vous pouvez définir ce paramètre

**[defaults]**
**log_path = /chemin/vers/votre/journal.log**

# Les Handlers

## Les handlers de Ansible

- Les **Handler** représentent une fonctionnalité importante dans Ansible qui permet de gérer les actions déclenchées en réponse à des changements dans les tâches exécutées dans les Playbooks

- Il ne faut pas confondre les **Handler** avec les tâches qui sont exécutées séquentiellement. Les **Handler** sont activés uniquement lorsqu'ils sont notifiés par d'autres tâches

- Voici quelques points clés à retenir sur les **Handler** dans Ansible :

**Déclenchement par les tâches :**

Lorsqu'une tâche modifie l'état du système d'une manière qui nécessite une action supplémentaire, elle peut notifier un ou plusieurs handlers pour effectuer ces actions.

**Définition des Handler :**

Les Handler sont définis dans la section handlers du **Playbook.** Chaque handler est associé à un nom unique et contient une ou plusieurs actions à exécuter en réponse à sa notification

## Les handlers de Ansible

### La notification:

Pour notifier un handler, une tâche utilise la directive notify avec le nom du handler

### Exécution différée:

Les handlers sont exécutés à la fin de l'exécution du playbook, après que toutes les tâches ont été traitées. Cela garantit que les handlers ne sont exécutés qu'une seule fois et à la fin de l'exécution du playbook, même s'ils sont notifiés plusieurs fois par différentes tâches

### Les performances:

Les handlers permettent d'optimiser les performances des playbooks en évitant des redémarrages ou des actions inutiles. Par exemple, plusieurs tâches peuvent notifier le même handler, mais celui-ci ne sera exécuté qu'une seule fois à la fin de l'exécution du playbook.

## Les handlers de Ansible

- Les **Handler** assurent que certaines tâches sont réussies tout d'abord pour continuer l'exécution du reste des tâches

La commande d'appel: ansible-playbook install_nginx.yaml --extra-vars "ansible_become_pass=test123++"

```
---
- name: Setup web servers with handler
  hosts: web_servers
  become: true
  become: root
  tasks:
    - name: Ensure Nginx is installed
      apt:
        name: nginx
        state: present
      notify: Restart Nginx

    - name: Ensure Nginx is running
      service:
        name: nginx
        state: started

    - name: Copy index.html
      copy:
        src: index.html
        dest: /var/www/html/
      notify: Restart Nginx

  handlers:
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
```

# Les variables

## Les variables de Ansible

- Les **Variables** sont des espaces réservés qui stockent des valeurs qui peuvent être utilisées dans les Playbook, les rôles et les tâches. Ils permettent des configurations dynamiques et rendent les playbooks plus réutilisables et adaptables.

- Utiliser la syntaxe {{ variable_name }} pour référencer une variable dans des Playbook, des modèles ou des fichiers de tâches

```
---
- name: exemple de variables
  hosts: ubuntu
  become: true
  vars:
     var1: Hello
     var2: World

  tasks:
   - name: echo text
     shell: echo "{{var1}} {{var2}}" > content.txt
```

**Définition dans le même fichier**

```
var1: Hello
var2: World
```

**vars.yml**

```
---
- name: exemple de variables
hosts: ubuntu tasks:
-    name: Chargement de variables
include_vars: file: vars.yml - name: Affichage de
message shell: echo "{{ var1 }} {{ var2 }}" >
/tmp/content.txt
```

**Définition dans  un fichier différent**

```
ansible-playbook test_variable.yml -e "var1=hello var2=world"
```

## Les niveaux de définitions des variables

**Inventaire :** Les variables peuvent être définies dans les fichiers d'inventaire à l'aide de variables d'hôte ou de groupe.

**Playbooks :** Les variables peuvent être définies au niveau du Playbook à l’aide d’un mot-clé **vars**

**Rôles :** Les variables peuvent être définies dans des fichiers de variables spécifiques au rôle situés dans le répertoire **vars**/de la structure du répertoire des rôles.

**Tâches :** les variables peuvent être définies directement dans les tâches à l'aide du module **set_fact**

**Ligne de commande :** Les variables peuvent également être définies directement au niveau de la ligne de commande en utilisant l'option **-e**.

## Les niveaux de définitions des variables

**Un chemin particulier:** Il est possible de pointer vers un fichier particulier qui contient les variables en utilisant le module **include_vars**

```
[defaults]

# Repertoire contenant les variables d'hôtes
host_vars = /chemin/vers/dossier_contient_fichiers_vars

# Repertoire contenant les variables de groupe
group_vars = /chemin/vers/dossier_contient_fichiers_group_vars
```

## Les variables d'hôte vs variables de groupe

Les variables d'hôte sont des variables spécifiques à un hôte particulier

```
# Inventaire
all:
  hosts:
    serveur1:
      ip_address: 192.168.1.10
      os: ubuntu
    serveur2:
      ip_address: 192.168.1.20
      os: centos
```

## Les variables d'hôte vs variables de groupe

Les variables de groupe sont des variables partagées entre plusieurs hôtes qui sont regroupés dans un groupe

```
# Inventaire
all:
  children:
    serveurs_web:
      hosts:
        serveur1:
        serveur2:

  vars:
    web_port: 80
    web_server_version: "2.4"
```

## Les variables de Ansible (Les niveaux de définitions )

**Inventaire :** Les variables peuvent être définies dans les fichiers d'inventaire à l'aide de variables d'hôte ou de groupe voici un exemple de définition de variable au niveau du fichier **/etc/ansible/hosts**

```
[ubuntu]
192.168.1.16
[ubuntu:vars]
valeur=8080
```

**Playbooks :** Les variables peuvent être définies au niveau du Playbook à l'aide d'un mot-clé **vars**

```
---
- name: Exemple de Playbook définissant des variables
  hosts: all
  vars:
  valeur: 'Hello'

  tasks:
    - name: Afficher la variable
      debug:
        msg: 'la valeur est {{ valeur }}'
```

Référence vers la variable

## Les variables au niveau des tâches

**Tâches :** Les variables peuvent être définies directement dans les tâches à l'aide du module **set_fact**

```
---
- name: Ajouter ou modifier server_name dans nginx.conf
  hosts: clients
  become: yes
  become_user: ubuntu
  tasks:
    - name: Definir une variable
      set_fact:
        valeur: 'test'
    - name: Utilisation de la variable
      debug:
        msg: '{{valeur}}'
```

## Les variables système

Les variables intégrées dans Ansible sont des variables prédéfinies qui fournissent des informations sur le système, le **Playbook** ou l'environnement dans lequel Ansible s'exécute

Ces variables peuvent être très utiles:

Pour les instructions conditionnelles

Le templating

Le débogage

Les variables intégrées peuvent être listées

**ansible -m debug -a "var=hostvars" localhost**

**ansible all -m ansible.builtin.setup**

```
---
- name: Ajouter ou modifier server_name dans nginx.conf
  hosts: clients
  become: yes
  become_user: jenkins
  tasks:
    - name: Utilisation de la variable
      debug:
        msg: 'Le nom d''hôte {{inventory_hostname}}'
```

# Le templating et jinaj2

**La flxibilité des Playbooks avec les templates et jinja2**

- La création de fichiers statiques pour chaque cas n'est pas une solution efficace dans certain cas. Jinja2 est un moteur de création de modèles utilisés par Ansible pour générer dynamiquement des prototypes basées sur des variables et des expressions au niveau des PlayBook

- Voici quelques notes

  - **{{ }}** : Ces doubles accolades sont les balises largement utilisées dans un fichier modèle, elles sont utilisées pour intégrer des variables et finalement imprimer leur valeur lors de l'exécution du code

  - **{% %}** : Ces accolades sont principalement utilisés pour les instructions de contrôle telles que les boucles et les instructions if-else

  - **{# #}** : Ces accolades désignent des commentaires qui décrivent une tâche.

- Les fichiers modèles portent l' extension **.j2** , ce qui implique que le modèle Jinja2 est utilisé

**Les fichiers templating à l'aide des varaibles de Playbooks avec jinja2**

- Pour comprendre voici un exemple

- Commencer par enregistrer ce contenu sous un fichier **test.j2**

Ce ci est un fichier template version_number et  server changent à partir du Playbook
La {{ version_number }} est executée sur le serveur {{ server }}!!!

- Exécuter ce Playbook

```
---
- hosts: ubuntu
  vars:
    version_number: 'v1.0.0'
    server: 'Ubuntu'
  tasks:
    - name: tester jinja 2
      template:
        src: test.j2
        dest: /home/ansuser/resultat.txt
```

## Les fichiers templating à l'aide des varaibles de Playbooks avec jinja2

- Voici un deuxième exemple **exemple2.j2**

```
La liste des légumes
{% for item in legumes %}
        {{ item }}
{% endfor %}
```

- Exécuter ce Playbook

```
---
- hosts: ubuntu
  vars:
    legumes: ['Piments','Tomates','Onions']

  tasks:
    - name: test de l'exemple 2 jija
      template:
        src: exemple2.j2
        dest: /home/ansuser/légumes.txt
```

**Les filtres de jinja2**

- Les filtres sont utilisés pour modifier l'apparence des données de sortie ou de formatage

- Par exemple, pour imprimer les valeurs de la liste précédente en majuscules à l'aide du modèle, dirigez l'élément variable vers l'argument ' upper ' comme indiqué : {{ item | upper }}

- Pour imprimer les valeurs de la liste précédente en miniscule à l'aide du modèle, dirigez l'élément variable vers l'argument ' lower ' comme indiqué : {{ item | lower }}

- Par exemple, pour imprimer la valeur minimale d'une liste, transmettez la liste entière au filtre ' min ' comme indiqué {{ 100,33,45,65,60,78 | min }} => 38

- Par exemple, pour imprimer la valeur maximale d'une liste, transmettez la liste entière au filtre ' max ' comme indiqué {{ 888,37,45,65,60,78 | max }} => 888

- Pour remplacer une chaîne par une nouvelle
  {{ 888,37,45,65,60,78 | replace("37","38") }} => 888,38,45,65,60,78

- Pour obtenir des valeurs uniques à partir d'une liste de valeurs en double dans un tableau
  {{ 1,1,1,2,2,3,3,4,4,5,5 | unique}} => 1,2,3,4,5

# Les modules

## Le module setup de ansible

- Le module **Setup** permet de collecter des informations sur les machines cibles
  Ansible  ubuntu –m setup

```
ansuser@PC2023:/etc/ansible$ ansible ubuntu -m setup
192.168.1.14 | SUCCESS => {
    "ansible_facts": {
        "ansible_all_ipv4_addresses": [
            "192.168.1.14"
        ],
        "ansible_all_ipv6_addresses": [
            "fe80::a00:27ff:fe03:9464"
        ],
        "ansible_apparmor": {
            "status": "enabled"
        },
```

- Avec le setup module peut connaître le type de l'os de la machine distante
  ansible ubuntu –m setup  -a "filter=*family*"

```
ansuser@PC2023:/etc/ansible$ ansible ubuntu -m setup -a "filter=*family*"
192.168.1.14 | SUCCESS => {
    "ansible_facts": {
        "ansible_os_family": "Debian",
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false
}
ansuser@PC2023:/etc/ansible$
```

**Module Command**

Ce module est utilisé pour exécuter n’importe quelle commande sur le système cible

```
- name: Création d'un répertoire
  command: mkdir /home/ubuntu/test-folder
```

**Module Shell**

Module Shell (shell) : Ce module est utilisé pour exécuter des commandes shell

```
- name: Exécuter une commande shell
  shell: echo "Hello, World!“ >> fichier_hello.txt
```

**Note:** La différence entre le module **command** et le module **shell**
Les opérations telles que les redirections (“<”, “>”), les pipes (“|”), et les opérateurs booléens (“&&”, “||”) ne fonctionneront pas avec le module **command**, ce pendant ce dernier est est généralement plus rapide que le module **shell**

**Les modules d'installation de packages**

Module Gestion de Paquets (package) : Il existe un module pour la plupart des gestionnaires de paquets populaires, tels que DNF, YUM et APT, qui vous permet d'installer n'importe quel paquet sur un système1.

```
- name: Installer la dernière version d'Apache et MariaDB
  dnf:
    name:
      - httpd
      - mariadb-server
    state: latest
```

**Module Service**

Le module service permet de démarrer, d'arrêter et de recharger les paquets installés exposés sous forme de service

```
  - name: Démarrer le service foo, basé sur le processus en cours d'exécution /usr/bin/foo
    service:
      name: foo
      pattern: /usr/bin/foo
      state: started
```

```
ansible-playbook arrete_nginx.yml --ask-become-pass
```

**Module File (file) :**

Ce module est utilisé pour gérer les fichiers et les répertoires

```
- name: Créer un répertoire
  file:
    path: /path/to/directory
    state: directory
```

**Module Copy :**

Ce module est souvent utilisé dans l'écriture de Playbooks lorsque nous voulons copier un fichier d'un serveur distant vers des nœuds de destination

```
- name: Copier un fichier
  copy:
    src: /home/knoldus/Personal/blogs/blog3.txt
    dest: /tmp
```

**Module Lineinfile (lineinfile) :**

Ce module est utilisé pour gérer les lignes dans les fichiers texte

```
- name: Ajouter une ligne à un fichier
  lineinfile:
    path: /etc/hosts
    line: '127.0.0.1 localhost'
```

**Module Get_url (get_url) :**

Ce module est utilisé pour télécharger des fichiers à partir d’URL

```
- name: Télécharger un fichier
  get_url:
    url: http://example.com/path/to/file
    dest: /path/to/destination
```

**Module Unarchive (unarchive) :**

Ce module est utilisé pour extraire des fichiers à partir d’archives

```
- name: Extraire une archive
  unarchive:
    src: /path/to/source
    dest: /path/to/destination
```

**Module User (user) :**

Ce module est utilisé pour gérer les utilisateurs sur le système cible

```
- name: Ajouter un utilisateur
  user:
    name: john
    state: present
```

**Module Group (group) :**

Ce module est utilisé pour gérer les groupes sur le système cible

```
- name: Ajouter un groupe
  group:
    name: admins
    state: present
```

**Module Git (git) :**

Ce module est utilisé pour cloner des dépôts git

```
- name: Cloner un dépôt git
  git:
    repo: https://github.com/example/repo.git
    dest: /path/to/destination
```

# Les conditions

## Le clause ignore_errors

- Le comportement par défaut de **Ansible** lorsqu'il rencontre une erreur est de ne pas continuer l'exécution du reste des tâches sauf si **ignore_errors: yes** est ajoutée à la fin de la tâche douteuse

```
---
- hosts: ubuntu
  #gather_facts: yes
  tasks:
  - name: "Copy a file"
    copy:
      src: /home/ansadmin/xyz.txt
      dest: /home/ansadmin
    ignore_errors: yes

  - name: "After error"
    debug: msg="Task after the error"
```

```
PLAY [ubuntu] ***********************************************************

TASK [Gathering Facts] **************************************************
ok: [192.168.1.20]

TASK [Copy a file] ******************************************************
An exception occurred during task execution. To see the full traceback, use
ing a module and expect the file to exist on the remote, see the remote_src
fatal: [192.168.1.20]: FAILED! => {"changed": false, "msg": "Could not find
 on the Ansible Controller.\nIf you are using a module and expect the file t
ote_src option"}
...ignoring

TASK [After error] ******************************************************
ok: [192.168.1.20] => {
    "msg": "Task after the error"
}

PLAY RECAP **************************************************************
192.168.1.20               : ok=3    changed=0    unreachable=0    failed=0
ed=1
```

## La clause register

- Le mot-clé register est utilisé pour stocker la sortie d'une tâche dans une variable. Cette variable peut ensuite être utilisée dans les tâches suivantes à diverses fins, comme la sortie de débogage ou les instructions conditionnelles.

- Voici un exemple simple :

```
- name: Test de register
  hosts: ubuntu
  tasks:

    - name: Exécuter une commande
      command:
        cmd: echo "Bonjour, Monde !"
      register: sortie_commande

    - name: Afficher la sortie de la commande
      debug:
        msg: "{{ sortie_commande.stdout }}"
```

```
PLAY [Test de register] ***********************************************

TASK [Gathering Facts] ************************************************
ok: [ubuntu]

TASK [Exécuter une commande] ******************************************
changed: [ubuntu]

TASK [Afficher la sortie de la commande] ******************************
ok: [ubuntu] => {
    "msg": "Bonjour, Monde !"
}

PLAY RECAP ************************************************************
ubuntu                     : ok=3    changed=1    unreachable=0    failed=0
```

## La clause when

- La clause **when** dans Ansible est utilisée pour contrôler si une tâche ou un rôle doit être exécuté en fonction d'une condition.

- La condition peut être une simple comparaison de variables, ou elle peut être une expression plus complexe

```
---

- name: Test Simple de When
  hosts: ubuntu
  tasks:

    - name: Vérifier si le système est Ubuntu
      debug:
        msg: "C'est un système Ubuntu."
      when: ansible_distribution == "Ubuntu"


    - name: Vérifier si le système est Centos
      debug:
        msg: "C'est un système Centos."
      when: ansible_distribution == "Centos"
```

```
PLAY [Test Simple de When] ******************************************************

TASK [Gathering Facts] **********************************************************
ok: [ubuntu]

TASK [Vérifier si le système est Ubuntu] ****************************************
ok: [ubuntu] => {
    "msg": "C'est un système Ubuntu."
}

TASK [Vérifier si le système est Centos] ****************************************
skipping: [ubuntu]

PLAY RECAP **********************************************************************
ubuntu                     : ok=2    changed=0    unreachable=0    failed=0    skipped=1
```

# Le débogage

## Le débogage des Playbooks Ansible

- Il est possible de déboguer les Playbook ansible avec le mot clé debug

```
---
- hosts: ubuntu
  #gather_facts: yes
  tasks:
  - name: "Copy a file"
    copy:
      src: /home/ansadmin/xyz.txt
      dest: /home/ansadmin
    ignore_errors: yes

  - name: "After error"
    debug: msg="Task after the error"
```

```
PLAY [ubuntu] ***********************************************************

TASK [Gathering Facts] **************************************************
ok: [192.168.1.20]

TASK [Copy a file] ******************************************************
An exception occurred during task execution. To see the full traceback, use
ing a module and expect the file to exist on the remote, see the remote_src
fatal: [192.168.1.20]: FAILED! => {"changed": false, "msg": "Could not find
 on the Ansible Controller.\nIf you are using a module and expect the file t
ote_src option"}
...ignoring

TASK [After error] ******************************************************
ok: [192.168.1.20] => {
    "msg": "Task after the error"
}

PLAY RECAP **************************************************************
192.168.1.20               : ok=3    changed=0    unreachable=0    failed=0
ed=1
```

# Les boucles

## Les boucles dans Ansible

- Il est possible de lancer une boucle dans Ansible avec la clause **with_items** et **with_sequence**

```
---
- hosts: ubuntu
  tasks:
  - name: Test de boucle
    debug: msg="{{item}}"
    with_items:
      - "Un"
      - "Deux"
      - "Trois"
```

**with_items equvalente à While**

```
---
- hosts: ubuntu
  tasks:
  - name: Test de boucle
    debug: msg=itération {{item}}
    with_sequence: start=20 end=25
```

**with_sequence equivalente à For**

# La sécurité

## Protéger les Playbook à l'aide de cryptage (ansible-vault)

- La commande **ansible-vault** permet de crypter le contenu d'un Playbook pour la protection
  **sudo ansible-vault encrypt légumes.yml**

```
ansuser@PC2023:~$ sudo cat légumes.yml
$ANSIBLE_VAULT;1.1;AES256
6338383264323339303735656461346330363061623464653133353536353638313462626661333 6
6663646162663333626435663937363630393738393663610a396531613938313464623937656431
6633383133393237353461646630333835613836353533353037303461313230623834303232346 5
3633663432393630360a6434333962396265646162336465383935306234383033663062303263 66
3166626466663236653935376330643262316339396139633137626466393330623064666263656 5
3233346333616439316133316430303863313130316338396664623530636161386562376338303 7
3634646639616664343136633762636430386639643634323936353339393366383334303564326 1
6330323231653035663133376237333764373235356230313864346330663465663336386435373 4
3262343030356432376261623837346362656633653534343430646435643663356531393462333 6
3163323136343234666162303837393231343435373666353831613736303431666266343435626 4
3962356534326166306266633163363339653737613364313331323631393936643036633664383 0
6263653630636630393431383836373838303137363337643732383638376535313030393639613 8
3235343736396365636266343038626333643637656536313836306464376666663963393836636 6
3736646637623434663664323866306663636161353037376635
```

- Pour éditer le fichier convenablement
  **sudo ansible-vault edit légumes.yml**

```yml
---
- hosts: ubuntu
  vars:
    legumes: ['Piments','Tomates','Onions']

  tasks:
    - name: test de l'exemple 2 jija
      template:
        src: exemple2.j2
        dest: /home/ansuser/légumes.txt
~
```

**Note:** Il faut créer initialement le Playbook avec la commande ansible-vault create

**Protéger les Playbook ansible en les cryptant en utilsant ansible-vault**

- Pour utiliser le playbook
  **sudo ansible-vault edit légumes.yml --ask-vault-pass**

- Pour utiliser le playbook il est également possible de décrypter le Playbook avant de l'utiliser
  **sudo ansible-vault decypt légumes.yml**

**Protéger les Playbook ansible en les cryptant en utilsant ansible-vault**

- Pour utiliser le playbook
  **sudo ansible-vault edit légumes.yml --ask-vault-pass**

- Pour utiliser le playbook il est également possible de décrypter le Playbook avant de l'utiliser
  **sudo ansible-vault decypt légumes.yml**

## Protéger les mots de passes et les tokens

- Il est conseillé de créer le  fichier pour stocker les données sensibles avec la commande

```
ansible-vault create credentials.yml
```

- Le fichier sera crypté dès qu'il sera fermé

```
# credentials.yml
database_username: myuser
database_password: mypassword
```

- Le fichier sera ensuite inclus dans le script du Playbook

```
- name: Exemple de playbook qui consomme des secrets
  hosts: localhost
  vars_files:
    - credentials.yml
  tasks:
    - name: Afficher le nom de la base de données et le mot de passe
      debug:
        msg: "Username: {{ database_username }}, Password: {{ database_password }}"
```

## Protéger les mots de passes et les tokens

- Il faut faire l'appel du playbook final

```
ansible-playbook playbook.yml --ask-vault-pass
```

# La structuration d'un projet Ansible

## Appler un playbook  à partir d'un autre playbook

- Importer playbook first et playbook second dans playbook main

```
---
- import_playbook:
first.yml
- import_playbook:
second.yml
```

**main.yml**

```
---
- name: first playbook
  hosts: all
  become: true


  tasks:
    - name: First task
      file:
        path: "first_file"
        state: touch
```

**first.yml**

```
---
- name: second playbook
  hosts: all
  become: true


  tasks:
    - name: First task
      file:
        path: "second_file"
        state: touch
```

**second.yml**

## Appler un playbook  à partir d'un autre playbook

- Importer  la tâche task1 et la tâche task2 second dans le playbook main

```
---
- name: main task
  hosts: all
  become: true

  tasks:
    - include: task1.yml
    - include: task2.yml
```

**main.yml**

```
---
- name: First task
  file:
    path: "first_file"
    state: touch
```

**task1.yml**

```
---
- name: Second task
  file:
    path: "second_file"
    state: touch
```

**task2.yml**

# Les rôles

## Les rôles Ansible

- Les rôles dans Ansible sont des unités de composition et de réutilisation du code, permettant d'organiser d'une manière modulaire les tâches, les variables, les fichiers et d'autres artefacts d'un Playbook.

- Les rôles simplifient la gestion des configurations en permettant de décomposer les tâches en fonctionnalités distinctes et réutilisables

```
ansadmin@machine1:~/playbooks/first_role$ tree
.
├── defaults
│   └── main.yml
├── handlers
│   └── main.yml
├── meta
│   └── main.yml
├── README.md
├── tasks
│   └── main.yml
├── tests
│   ├── inventory
│   └── test.yml
└── vars
    └── main.yml
```

## Les rôles Ansible

- Création d'un nouveau rôle :
  Pour créer un nouveau rôle, vous pouvez utiliser la commande **ansible-galaxy**, voici un exemple

```
ansadmin@machine1:~/playbooks$ ansible-galaxy init nginx_role

- Role nginx_role was created successfully
ansadmin@machine1:~/playbooks$
ansadmin@machine1:~/playbooks$
```

**Dans defaults/main.yml:** Il est possible de définir des variables par défaut, telles que le port d'écoute par défaut

**Dans tasks/main.yml:** Il est possible de définir les tâches à ce niveau

**Dans handlers/main.yml:** Il est possible de définir un gestionnaire pour redémarrer u service après la modification de la configuration par exemple

**Dans vars/main.yml:** Il est possible de définir des variables globales utilisées dans le rôle d'une manière générale

## Les rôles Ansible

- L'utilisation du rôle se fait de la manière suivante, il faut créer un **Playbook** et appeler le rôle par son nom

```yaml
---
- name: Playbook to configure Nginx server
  hosts: ubuntu
  roles:
    - nginx_role
```

## Les rôles Ansible

- Mettre le code suivant dans **main.yml** au niveau du dossier **tasks**

```
---
- name: Install web server packages
  apt:
    name: apache2
    state: present

- name: Copy configuration file
  template:
    src: templates/httpd.conf.j2
    dest: /etc/apache2/httpd.conf

- name: Restart web server
  service:
    name: apache2
    state: restarted
```

## Les rôles Ansible

- Mettre le script suivant dans **main.yml** au niveau dossier **handlers**

```
---
- name: Restart web server
  service:
    name: apache2
    state: restarted
```

- Définir les variables suivantes au niveau du **main.yml** dans le dossier **vars**

```
---
http_port: 80
http_conf_file: /etc/apache2/httpd.conf
```

- Définir les variables suivantes au niveau du **main.yml** dans le dossier **defaults**

```
---
http_port: 80
http_conf_file: /etc/apache2/httpd.conf
```

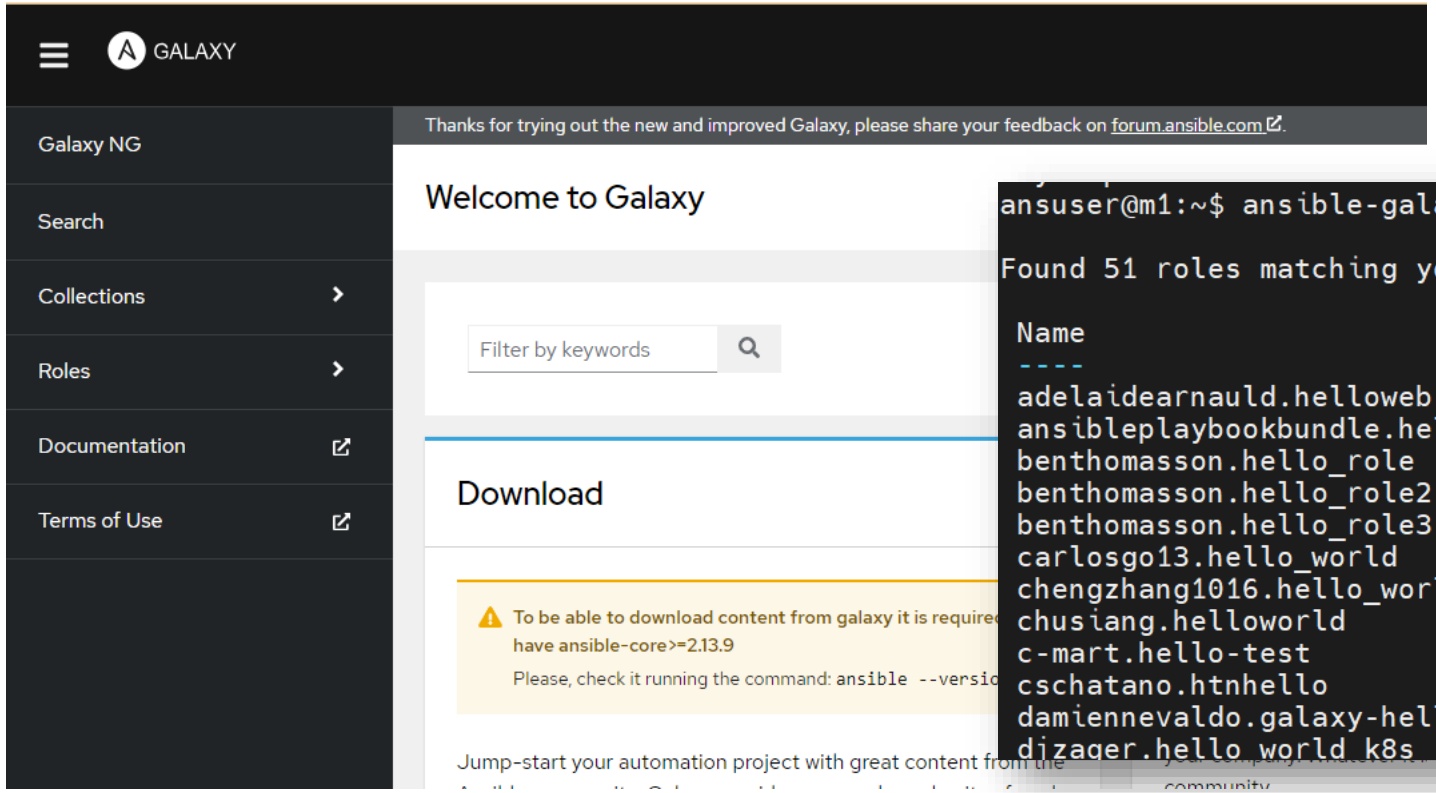
## Les rôles Ansible

- Pour appeler le rôle, il faut l'inclure tout simplement dans l'élément roles

```
---
- name: Configure web server
  hosts: all
  become: true
  roles:
    - webserver
```

## Chercher un rôle dans Ansible Galaxy

- Ansible Galaxy est une plateforme communautaire permettant de partager et de trouver des rôles Ansible




```
ansuser@m1:~$ ansible-galaxy search hello

Found 51 roles matching your search:

 Name                                        Description
 ----                                        -----------
 adelaidearnauld.helloweb                    your description
 ansibleplaybookbundle.hello-world-apb       Deploys hello-world web appl
 benthomasson.hello_role                     Hello World role
 benthomasson.hello_role2                    Hello World role 2
 benthomasson.hello_role3                    Hello World role
 carlosgo13.hello_world                      An Ansible role to install a
 chengzhang1016.hello_world_db_apb           A sample APB which deploys H
 chusiang.helloworld                         An Ansible role example for
 c-mart.hello-test                           test
 cschatano.htnhello                          Getting started
 damiennevaldo.galaxy-helloweb               your description
 dizaqer.hello_world_k8s                     Manage hello-world applicati
```

- Pour installer un rôle, il faut le télécharger

**Publier un rôle vers Ansible Galaxy**

- Ansible Galaxy permet de partager des rôles Ansible pour publier un rôle il faut avoir un compte https://galaxy.ansible.com/

- Après la création du rôle, il faut ajouter dans le dossier meta les informations nécessaires qui décrient le rôle

**galaxy_info**:
  **author**: Bechir
  **description**: Test de publication de rôle
  **company**: emraude it
  **license**: license (GPLv2, CC-BY, etc)
  **min_ansible_version**: 1.2

- Packager le rôle  tar –czf <nom du rôle>.tar.gz <nom du rôle>
- Cliquer sur le bouton "Publier" dans le coin supérieur droit
- Remplir les informations requises dans le formulaire de publication et publier
- Vérifier l'installation du rôle

## Les rôles

**Rôles :** Les variables peuvent être définies dans des fichiers de variables spécifiques aux rôles situés dans le répertoire vars/de la structure du répertoire des rôles.

```
ansadmin@machine1:~/playbooks/first_role$ tree
.
├── defaults
│   └── main.yml
├── handlers
│   └── main.yml
├── meta
│   └── main.yml
├── README.md
├── tasks
│   └── main.yml
├── tests
│   ├── inventory
│   └── test.yml
└── vars
    └── main.yml
```